CAN-SENTINEL

Team Work Distribution & Remote Collaboration Plan

Project: Automotive ECU Intrusion Detection System

Purpose: divide the prototype into independent modules so each team member can develop and test their work remotely, then integrate the modules through a shared GitHub repository.

Note: The allocation below is a proposed plan. Confirm it against the internship mentor's official assignment and your team's actual member count before announcing final ownership.

1. Proposed module responsibilities

Member 1 — CAN Data Collection & Simulation

Objective: Provide CAN messages for analysis.

- Research CAN message structure and agree on a shared data format.
- Generate simulated CAN traffic or prepare a suitable public dataset.
- Include normal traffic and clearly labelled simulated suspicious examples.
- Export sample data to CSV and validate required fields.

Deliverables: Python generator/loader, sample CSV, validation tests, README.

Definition of done: Other modules can load the data using the documented format.

Member 2 — Intrusion Detection Engine

Objective: Flag potentially suspicious CAN traffic patterns.

- Implement configurable rules for unusually high message frequency.
- Flag CAN IDs outside a configured allowlist and malformed records.
- Test rules against normal and simulated suspicious traffic.
- Document thresholds, assumptions, and limitations.

Deliverables: Python detection module, rule configuration, unit tests, detection documentation.

Definition of done: The module reads the agreed input format and produces structured detection events.

Member 3 — Security Analysis & Alert Management

Objective: Turn detection events into consistent, traceable alerts.

- Define documented Low/Medium/High severity rules.
- Record timestamp, CAN ID, event reason, severity, and status.
- Store alert history in JSON or CSV and handle duplicates where appropriate.
- Test the module using sample detection events.

Deliverables: Alert manager, severity configuration, sample logs, tests, README.

Definition of done: Detection events are converted into alerts the dashboard can display.

Member 4 — Dashboard & Integration

Objective: Present traffic statistics and security alerts in a usable interface.

- Build a dashboard using Streamlit or an agreed framework.
- Display messages processed, suspicious events, severity counts, recent alerts, and traffic frequency.
- Add practical filters such as CAN ID, severity, or time.
- Integrate the modules and test the end-to-end prototype.

Deliverables: Dashboard app, integration code, screenshots, tests, run instructions.

Definition of done: Dashboard displays results from connected modules, not only hard-coded demo values.

2. How the modules connect

Member 1 Data Collection	**Member 2** Detection Engine	**Member 3** Alert Management	**Member 4** Dashboard
CAN messages / dataset	Detection events	Structured alerts	Displayed results

Integration rule: agree on input/output schemas before implementation. Member 4 can use sample JSON/CSV fixtures to build the interface before the other modules are finished.

3. Day 1 tasks

Owner	Today's starting task	Expected result
Member 1	Define CAN message fields and prepare a tiny sample dataset.	Sample CSV + format note
Member 2	Write initial detection-rule specification and output schema.	Detection specification
Member 3	Define alert fields and severity criteria.	Sample JSON alerts + criteria
Member 4	Create the dashboard skeleton using sample data.	First dashboard prototype

4. Remote collaboration rules

- Use one shared GitHub repository and a separate feature branch for each module.
- Each member commits their own changes and opens a pull request for review before merging into main.
- Every module must include a README with setup, execution, inputs, outputs, and limitations.
- Share progress and blockers regularly; document decisions in the repository.
- Do not upload credentials, confidential internship material, private vehicle data, or other restricted information.

5. Suggested four-week plan

Week	Focus	Expected result
1	Requirements, shared formats, sample data, initial prototypes	Clear interfaces and starter modules
2	Independent module development and unit tests	Four testable modules
3	Integration and bug fixing	End-to-end prototype
4	Validation, documentation, report, presentation, demo	Final submission package

6. Message to share with the team

Hey team!

We'll split CAN-Sentinel into four proposed modules so everyone can develop independently while working remotely:
1. CAN Data Collection & Simulation
2. Intrusion Detection Engine
3. Security Analysis & Alert Management
4. Dashboard & Integration

Before coding, we'll agree on shared input/output formats. Each member should work on their branch, test their module,

and submit source code, a README, and test results. Today's goal is to define interfaces and begin the initial tasks. Please share progress and blockers early. Let's build this step by step!

Technical note: this is a prototype plan. Use simulated traffic or an appropriate public dataset. An anomaly is not proof of an attack, and the prototype should not claim to prevent attacks unless that capability is implemented and tested.